



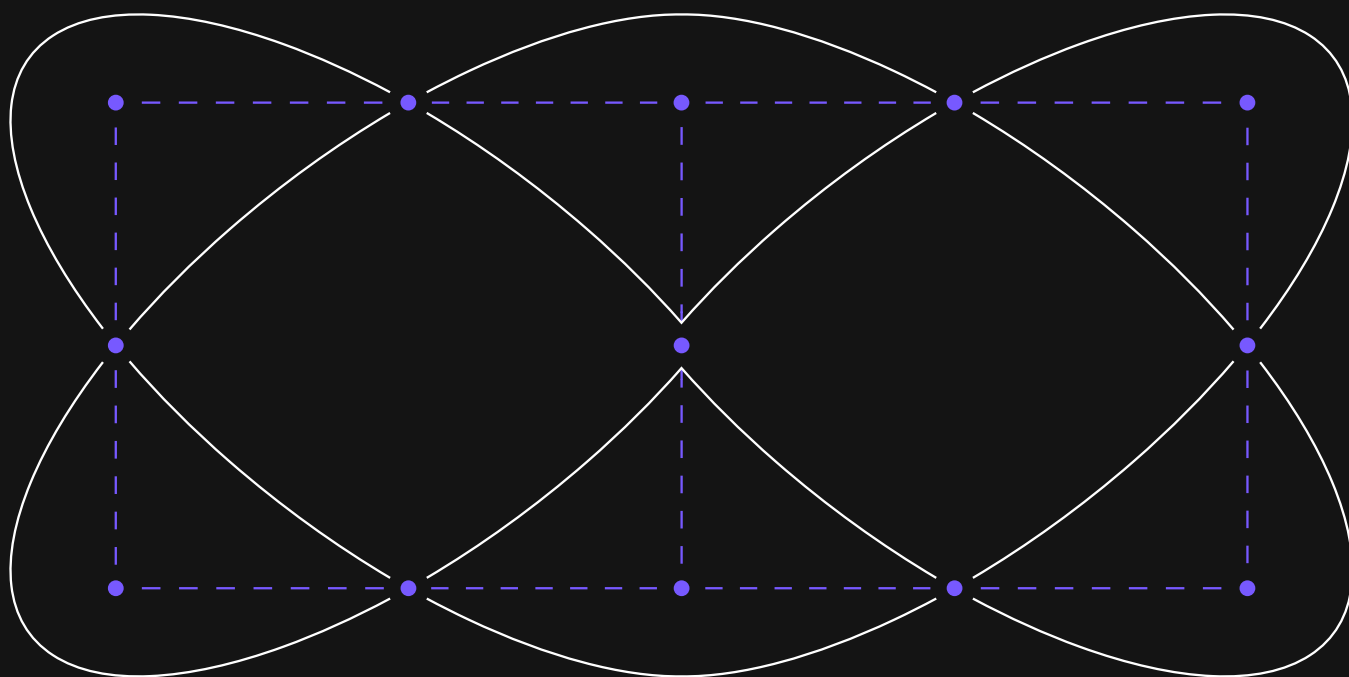
Машинное обучение

Лонгрид

Методы оптимизации

01

Повторение линейной алгебры и сходимость последовательностей



Нормы и спектр

Контекст

Любой метод оптимизации — это последовательность точек $x_0, x_1, x_2, \dots$, приближающих минимум некоторой функции f , и почти всё, что мы будем говорить о таких последовательностях, упирается в три вещи: чем измерять расстояние до решения, как устроена матрица вторых производных f и как эффективно вычислять саму функцию и её производные — что зачастую сводится к эффективности матрично-векторных операций. Первая вещь — это нормы, вторая — спектр, третья — матричные вычисления. Именно поэтому курс начинается не с методов, а с повторения линейной алгебры: скорость градиентного спуска выражается через собственные значения гессиана, скорость метода Ньютона — через его обусловленность, а вся геометрия «оврагов», в которых методы застревают, — это геометрия эллипсоидов, порождённых сингулярным разложением.

Применение

Практическая польза прямая: научившись читать спектр матрицы, ты сможешь до запуска метода сказать, сойдётся ли он за десять итераций или за десять тысяч, стоит ли масштабировать переменные и какой предобуславливатель имеет смысл. Те же самые сингулярные числа управляют сжатием данных, устойчивостью численного решения линейных систем и малоранговыми приближениями, на которых держится, например, дообучение больших языковых моделей.

Задача лонгрида

Разберём инструменты измерения: нормы векторов и матриц, скалярные произведения, спектральное и сингулярное разложения, отношение Рэлея и число обусловленности — и посмотрим, как всё это выглядит геометрически.

Краткий план

- Нормы и скалярные произведения
- Спектр, сингулярные числа и геометрия

Три величины, которые дальше будут встречаться постоянно:



$$\|A\|_2 = \sigma_{\max}(A), \quad \|A\|_F^2 = \sum_i \sigma_i^2(A), \quad \kappa(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}.$$

Первая — максимальное растяжение, вторая — «суммарная энергия» матрицы, третья — **число обусловленности**, мера того, насколько сильно матрица искажает пропорции.

Геометрический смысл SVD исчерпывающий: любое линейное отображение — это поворот, растяжение вдоль осей и ещё один поворот. Единичная сфера переходит в эллипсоид с полуосями $\sigma_1, \dots, \sigma_n$, направленными вдоль столбцов U . Тогда $\sigma_{\max}$ — длина самой длинной полуоси, $\sigma_{\min}$ — самой короткой, а $\kappa(A)$ — отношение этих длин, то есть «вытянутость» эллипсоида.

ПРИМЕР 1

Линии уровня функции $f(x) = \frac{1}{2}x^T A x$ для $A \succ 0$ — эллипсоиды с полуосями, пропорциональными $1/\sqrt{\lambda_i}$. При $A = \text{diag}(1, 100)$ они вытянуты в сто раз, и антиградиент в точке $(1, 1)$ смотрит почти перпендикулярно направлению на минимум: градиентный спуск будет «зигзагом» ползти по дну «оврага». Число обусловленности — это ровно количественная мера этой беды.

1. ЕСЛИ КРАТКО

- Норма Фробениуса — это евклидова длина матрицы, разложенной в вектор; она порождена скалярным произведением $\langle A, B \rangle = \text{tr}(A^T B)$ и равна $\sqrt{\sum_i \sigma_i^2}$.
- Операторная норма измеряет максимальное растяжение вектора; для $p = 2$ она равна $\sigma_{\max}(A)$, и именно она согласована с векторной нормой, что делает её рабочим инструментом оценок.
- Спектральное разложение существует для симметричных матриц, сингулярное — для любых; собственные и сингулярные числа совпадают по модулю только в симметричном случае.
- Геометрия SVD: сфера $\rightarrow$ эллипсоид с полуосями σ_i . Число обусловленности $\kappa(A) = \sigma_{\max}/\sigma_{\min}$ — вытянутость этого эллипсоида и одновременно главный предиктор скорости методов первого порядка.

ЗАДАЧА 1

Докажи, что $\|A\|_F^2 = \text{tr}(A^T A) = \sum_i \sigma_i^2(A)$. Выведи отсюда, что $\|UAV\|_F = \|A\|_F$ и $\|UAV\|_2 = \|A\|_2$ для любых унитарных $U, V \in \mathbb{U}^n$.

ЗАДАЧА 2

Через собственные значения вырази определитель и след:

$$\det A = \prod_i \lambda_i, \quad \text{tr } A = \sum_i \lambda_i.$$

Геометрически $|\det A| = \prod_i \sigma_i$ — коэффициент изменения объёма при отображении $x \mapsto Ax$. Если $\sigma_{\min} = 0$, объём схлопывается: матрица вырождена.

ЗАДАЧА 3

Для операторной нормы $\|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2$ докажи, что $\|A\|_2 = \sigma_{\max}(A)$.

ЗАДАЧА 4

Покажи цепочку неравенств

$$\sigma_{\max}(A) \leq \|A\|_F \leq \sqrt{r} \sigma_{\max}(A), \quad r = \text{rank } A.$$

В каких случаях достигаются равенства?

ЗАДАЧА 5

Пусть Q — унитарная матрица. Докажи, что

$$\|Qx\|_2 = \|x\|_2, \quad \sigma_i(QA) = \sigma_i(A), \quad |\lambda_i(Q)| = 1.$$

ЗАДАЧА 6

Вырази через собственные значения величины $\det A$, $\text{tr } A$, $\det(I + A)$. Найди $\det(I + uv^T)$ и все собственные значения матрицы uv^T .

ЗАДАЧА 7

Пусть $A = U\Sigma V^T$. Опиши образ единичной сферы $\{x : \|x\|_2 = 1\}$ под действием A . Нарисуй картинку для

$$A = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

Где на рисунке видны σ_1, σ_2 , где — число обусловленности, где — собственные векторы?

Матричные вычисления

Контекст

В теории все формулы линейной алгебры равноправны: $(AB)x$ и $A(Bx)$ — это один и тот же вектор. На практике первое выражение при $n = 10^4$ считается сутки, а второе — доли секунды. Методы оптимизации живут в мире, где n измеряется миллионами и миллиардами, поэтому вопрос «В каком порядке умножать и что именно мы обязаны хранить в памяти?» перестаёт быть техническим и становится содержательным: он определяет, какие методы вообще применимы.

Применение

Освоив этот материал, ты научишься оценивать сложность выражения по его виду, не запуская код; видеть, когда матрицу можно не хранить целиком; понимать, почему современные методы первого порядка устроены именно так, а не иначе. Это ровно та оптика, которая позволяет запускать PageRank на графе из миллиарда вершин и дообучать модели с миллиардами параметров на одной видеокарте.

Задача лонгрида

Разберём язык индексной записи, правила дешёвых матричных вычислений и главную абстракцию курса — матрицу как оракул умножения на вектор.

Краткий план

- [Эйнштейновская нотация](#)
- [Порядок операций и структура матрицы](#)
- [Матрица как оракул: MatVec](#)

1. ЭЙНШТЕЙНОВСКАЯ НОТАЦИЯ

Договоримся выписывать выражения поэлементно и опускать знаки суммы: индекс, встречающийся в произведении дважды и отсутствующий в левой части, считается просуммированным.

ПРИМЕР 2

$$C_{ij} = A_{ik} B_{kj} \text{ — матричное умножение, } s = A_{ii} \text{ — след,}$$

$$y_i = A_{ij} x_j \text{ — умножение на вектор.}$$

Такая запись реализована в `numpy` (а также в `torch` и `jax`) функцией `einsum`: строка `'ik,kj->ij'` перечисляет индексы аргументов и результата, а всё, что в результат не попало, суммируется.

ЗАМЕЧАНИЕ

Польза от такой нотации двойная. Во-первых, одна запись покрывает след, транспонирование, свёртки, батчевые и тензорные операции, где привычная «матричная» нотация уже неудобна или попросту не работает. Во-вторых, из неё сразу видно, сколько индексов пробегается, а значит, и сколько операций стоит наивное вычисление — и насколько его можно удешевить, выбрав правильный порядок свёртки.

Индексная запись естественно продолжается на тензоры. Плотный тензор размера $n \times n \times n$ требует n^3 чисел, что уже при $n = 1000$ неподъёмно; вместо этого используют структурированные форматы.

ЗАДАЧА 1

Запиши через эйнштейновское суммирование (`einsum`) следующие операции:

$$\text{tr}(A), \quad \text{diag}(A), \quad A^\top B, \quad x^\top A y, \quad uv^\top,$$

а также батчевое умножение $C_{bij} = \sum_k A_{bik} B_{bkj}$.

ЗАДАЧА 2

Рассмотрим выражение

$$\sum_{i, j, k, l} A_{ij} B_{jk} C_{kl} D_{li}, \quad A, B, C, D \in \mathbb{R}^{n \times n}.$$

Сколько операций потребует «лобовой» перебор всех индексов? Найди как можно лучший порядок свёртки и его сложность.

ЗАДАЧА 3

Канонический (CP) тензорный формат задаётся как $T_{ijk} = \sum_{s=1}^r a_{is} b_{js} c_{ks}$. Сколько чисел хранит плотный тензор размера $n \times n \times n$ и сколько — его CP-разложение ранга r ? Посчитай для $n = 100, r = 10$.

2. ПОРЯДОК ОПЕРАЦИЙ И СТРУКТУРА МАТРИЦЫ



Умножение матриц ассоциативно, но не «асимптотически безразлично». Расстановка скобок меняет сложность на порядки, и почти всегда правильный ответ — «сначала умножить на вектор».

ПРИМЕР 3

Пусть $A, B \in \mathbb{R}^{n \times n}, x \in \mathbb{R}^n$. Вычисление $(AB)x$ требует $O(n^3)$ операций (матричное произведение) плюс $O(n^2)$; вычисление $A(Bx)$ — всего $2n^2$ операций. При $n = 10^4$ разница составляет примерно $n/2 = 5000$ раз.

Второй сюжет — структура. Матрица, заданная формулой, часто не должна храниться поэлементно: диагональную задаёт вектор из n чисел, матрицу ранга один uv^T — два вектора, разрежённую — список нулей. Наивная материализация такой матрицы уничтожает выигрыш.

ПРИМЕР 4

Умножение DA для $D = \text{diag}(d)$ — это масштабирование строк A , то есть $O(n^2)$ операций. Но код вида `np.diag(d) @ A` сначала строит плотную матрицу $n \times n$, а затем выполняет полноценное умножение за $O(n^3)$: цена ошибки — множитель n .

ЗАМЕЧАНИЕ

Тот же принцип позволяет обращаться структурированные матрицы, не обращая их. Формула Шермана — Моррисона

$$(I + uv^T)^{-1} = I - \frac{uv^T}{1 + v^T u}$$

решает систему с матрицей $I + uv^T$ за $O(n)$ операций вместо $O(n^3)$.

3. МАТРИЦА КАК ОРАКУЛ: MATVEC

Умножение матрицы на вектор ($\text{MatVec}, x \mapsto Ax$) — базовая операция всех итерационных методов. Степенной метод, метод сопряжённых градиентов, крыловские методы и градиентный спуск для квадратичной функции не делают с матрицей ничего, кроме этого.

ОПРЕДЕЛЕНИЕ 1

Оракул — чёрный ящик, который по запросу x возвращает значение Ax (или в общем случае значение функции, её градиента и тому подобное). Сложность метода

в оракульной модели измеряется числом обращений к оракулу, а не числом арифметических операций.



Матрица — это не таблица чисел, а способ действовать на вектор. Если ты умеешь вычислять Ax , тебе не нужна сама A .

Стоимость одного обращения зависит от структуры и может быть много меньше $O(n^2)$:

- диагональная матрица — $O(n)$;
- матрица ранга r вида UV^T , $U, V \in \mathbb{R}^{n \times r}$ — $O(nr)$;
- разрежённая матрица — $O(\text{nnz})$, где nnz — число ненулей;
- произвольная плотная — $O(n^2)$.

Сравни это с $O(n^3)$ у прямых методов решения $Ax = b$ и с необходимостью хранить n^2 чисел — и станет понятно, почему при больших n альтернативы итерационным методам просто нет.

ЗАДАЧА 4

Даны $A, B \in \mathbb{R}^{n \times n}$ и $x \in \mathbb{R}^n$. Сравни число арифметических операций для вычисления $(AB)x$ и $A(Bx)$. Во сколько раз они отличаются при $n = 10^4$?

ЗАДАЧА 5

Пусть матрица A доступна только через оракул $x \mapsto Ax$ и каждый его вызов стоит единицу. Как за n вызовов восстановить всю матрицу A ? Как за один вызов получить вектор сумм строк? Как получить j -й столбец?

ЗАДАЧА 6

Оцени стоимость одного умножения матрицы на вектор для следующих матриц:

- диагональная;
- вида $D + UV^T$, где $U, V \in \mathbb{R}^{n \times r}$;
- разрежённая с nnz ненулевыми элементами;
- произвольная плотная.

Сравни полученные оценки со сложностью $O(n^3)$ решения системы $Ax = b$.

Скорости сходимости и степенной метод

Контекст

Метод оптимизации почти никогда не даёт точный ответ за конечное число шагов — он строит последовательность приближений. Поэтому сравнивать методы можно только по одному признаку: как быстро убывает ошибка $r_k = \|x_k - x^*\|$. Классификация скоростей сходимости — это общий язык всего курса: фразы «градиентный спуск сходится линейно» и «метод Ньютона — квадратично» должны читаться как точные количественные утверждения, из которых сразу следует число итераций до нужной точности.

Применение

Практическая польза видна на числах: методу со скоростью $r_{k+1} \leq 0,9r_k$ нужно около трёхсот итераций до точности 10^{-15} , а квадратично сходящемуся — меньше шести. Умение по виду оценки предсказать это различие экономит часы вычислений и подсказывает, когда стоит переключиться на более дорогой по шагу, но более

быстрый метод. В конце соберём всё вместе на степенном методе и PageRank — алгоритме, который в своё время сделал Google.

Задача лонгрида

Введём шкалу скоростей и два инструмента их проверки, а затем разберём степенной метод и его применение к ранжированию веб-страниц.

Краткий план

- [Классификация скоростей сходимости](#)
- [Степенной метод](#)
- [PageRank: зачем это нужно](#)

1. КЛАССИФИКАЦИЯ СКОРОСТЕЙ СХОДИМОСТИ

Пусть $x_k \rightarrow x^*$ и $r_k = \|x_k - x^*\|$.

ОПРЕДЕЛЕНИЕ 2

Сходимость называется:

- **линейной** со знаменателем $q \in (0, 1)$, если $r_{k+1} \leq q r_k$ (ошибка убывает как геометрическая прогрессия);
- **сублинейной**, если $r_{k+1}/r_k \rightarrow 1$ (например, $r_k = 1/k$);
- **сверхлинейной**, если $r_{k+1}/r_k \rightarrow 0$;
- **квадратичной**, если $r_{k+1} \leq C r_k^2$.

ЗАМЕЧАНИЕ

Практический смысл шкалы в том, сколько верных знаков добавляет одна итерация. При линейной сходимости — фиксированную долю знака; при квадратичной число верных знаков удваивается на каждом шаге. Отсюда и разница между сотнями итераций и единицами.

Проверять тип сходимости можно двумя способами.

ОПРЕДЕЛЕНИЕ 3

Тест отношений: вычисляется $\lim_{k \rightarrow \infty} r_{k+1}/r_k = q$. **Тест корней:** вычисляется $\limsup_{k \rightarrow \infty} r_k^{1/k} = q$. В обоих случаях $q \in (0, 1)$ означает линейную сходимость, $q = 0$ — сверхлинейную, $q = 1$ — сублинейную.



Тест корней сильнее теста отношений: он использует $\limsup$ и потому работает для последовательностей с нерегулярным поведением, где отношение r_{k+1}/r_k вовсе не имеет предела. Обратное неверно: если предел отношений существует, тест корней даёт тот же ответ.

Простейший пример, на котором вся эта шкала оживает, — метод простой итерации для линейной системы.

ОПРЕДЕЛЕНИЕ 4

Метод простой итерации (метод Рундсона) для системы $Ax = b$ с $A \succ 0$ — итерация

$$x_{k+1} = x_k - \tau(Ax_k - b), \quad \tau > 0.$$

Это в точности градиентный спуск с постоянным шагом τ для функции $f(x) = \frac{1}{2}x^\top Ax - b^\top x$.

Вычитая из обеих частей решение x^* , получаем $x_{k+1} - x^* = (I - \tau A)(x_k - x^*)$: ошибка на каждом шаге умножается на одну и ту же матрицу. Значит, сходимость линейная со знаменателем $q(\tau) = \|I - \tau A\|_2 = \max_i |1 - \tau \lambda_i(A)|$ и всё определяется спектром A .

ЗАМЕЧАНИЕ

Оптимальный шаг $\tau^* = \frac{2}{\mu + L}$ (здесь $\mu = \lambda_{\min}$, $L = \lambda_{\max}$) даёт

$$q^* = \frac{L - \mu}{L + \mu} = \frac{\kappa(A) - 1}{\kappa(A) + 1}.$$

При $\kappa(A) = 1$ метод сходится за один шаг, при $\kappa(A) = 100$ знаменатель уже $\approx 0,98$. Число обусловленности буквально скорость метода.

2. СТЕПЕННОЙ МЕТОД

Самый простой итерационный метод линейной алгебры находит главный собственный вектор, пользуясь одним лишь оракулом MatVec.

ОПРЕДЕЛЕНИЕ 5

Степенной метод для матрицы A — итерация

$$x_{k+1} = \frac{Ax_k}{\|Ax_k\|}, \quad x_0 \text{ — произвольный начальный вектор.}$$

ТЕОРЕМА 1

Пусть $A = A^T$, её собственные значения удовлетворяют $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$, собственные векторы q_i ортонормированы, а начальный вектор $x_0 = \sum_i c_i q_i$ имеет $c_1 \neq 0$. Тогда для угла θ_k между x_k и q_1 выполнено

$$\tan \theta_k \leq \left| \frac{\lambda_2}{\lambda_1} \right|^k \tan \theta_0.$$

Это в точности линейная сходимость со знаменателем $q = |\lambda_2/\lambda_1|$ — величиной, которую называют **спектральным зазором**. Чем ближе второе собственное значение к первому, тем медленнее метод: при $q = 0,99$ на каждый десятичный знак точности уходит примерно 230 итераций.

ЗАМЕЧАНИЕ

Идея степенного метода тривиальна: разложив x_0 по собственному базису, видим, что $A^k x_0 = \sum_i c_i \lambda_i^k q_i$. Компонента с наибольшим $|\lambda_i|$ растёт быстрее всех и постепенно подавляет остальные; нормировка нужна лишь для того, чтобы вектор не улетал на бесконечность или к нулю.

3. PAGERANK: ЗАЧЕМ ЭТО НУЖНО

Задача ранжирования веб-страниц ставится так: есть ориентированный граф, вершины — страницы, рёбра — гиперссылки. Нужно приписать каждой странице число — её «важность» — так, чтобы поисковая выдача сортировалась по этому числу. Наивная идея «важность = число входящих ссылок» плоха: её легко накрутить, создав тысячу пустых страниц-однодневок. Идея PageRank рекурсивна: *страница важна, если на неё ссылаются важные страницы*, причём каждая страница делит свою важность поровну между всеми, на кого ссылается сама.

Формально: пусть $\pi_i \geq 0$ — важность страницы i , $\sum_i \pi_i = 1$, и пусть d_j — число исходящих ссылок страницы j . Тогда требование выше записывается как

$$\pi_i = \sum_{j \rightarrow i} \frac{\pi_j}{d_j}, \quad \text{то есть} \quad \pi = P\pi, \quad P_{ij} = \begin{cases} 1/d_j, & j \rightarrow i, \\ 0 & \text{иначе.} \end{cases}$$

Матрица P стохастична по столбцам, а искомый вектор π — её собственный вектор с собственным значением 1.

У этой постановки два дефекта. Во-первых, у графа могут быть «висячие» вершины (страницы без исходящих ссылок) и замкнутые группы страниц, ссылающихся только друг на друга, — в них вся важность «утрачивается» и накапливается. Во-вторых, собственный вектор, отвечающий $\lambda = 1$, может быть не единственным. Оба дефекта лечатся одной и той же модификацией — моделью **random surfer**: с вероятностью α пользователь переходит по случайной ссылке текущей страницы, а с вероятностью $1 - \alpha$ телепортируется на случайную страницу всего интернета:

$$G = \alpha P + \frac{1 - \alpha}{n} ee^\top, \quad \alpha = 0,85, \quad e = (1, \dots, 1)^\top.$$

ТЕОРЕМА 2

Перрон — Фробениус

Если все элементы стохастической по столбцам матрицы G строго положительны, то её главное собственное значение $\lambda_1 = 1$ простое, соответствующий собственный вектор положителен и единствен, а для остальных собственных значений выполнено $|\lambda_2| \leq \alpha$.

Вектор π — это стационарное распределение цепи Маркова, то есть доля времени, которую random surfer проводит на каждой странице.



Остаётся вычислительный вопрос: как найти π при $n \sim 10^9$? Прямые методы ($O(n^3)$) и явное хранение G ($n^2 \sim 10^{18}$ чисел) исключены полностью. Зато у нас есть степенной метод, которому от матрицы нужно ровно одно — умение умножать её на вектор. А произведение Gx считается за $O(\text{nnz}(P) + n)$, потому что вторая компонента G — матрица ранга один:

$$Gx = \alpha Px + \frac{1 - \alpha}{n} e(e^\top x).$$

4. ЕСЛИ КРАТКО

- Скорость сходимости — это характеристика убывания $r_k = \|x_k - x^*\|$: сублинейная, линейная, сверхлинейная, квадратичная. Она напрямую переводится в число итераций до заданной точности.
- Тест корней сильнее теста отношений: он даёт ответ там, где отношение r_{k+1}/r_k не имеет предела.
- Степенной метод сходится линейно со знаменателем $|\lambda_2/\lambda_1|$ и требует от матрицы только операции MatVec.
- PageRank — это степенной метод для матрицы $G = \alpha P + \frac{1 - \alpha}{n} ee^\top$. Он работает при $n \sim 10^9$ ровно потому, что G никогда не строится явно: разрежённая часть плюс поправка ранга один дают Gx за $O(\text{nnz}(P) + n)$.

ЗАДАЧА 1

1. Определи тип сходимости (сублинейная, линейная, сверхлинейная, квадратичная) для последовательностей

$$r_k = \frac{1}{k}, \quad r_k = 0,5^k, \quad r_k = \frac{1}{k!}, \quad r_k = 0,5^{2^k}.$$

2. Пусть теперь $r_k = (2 + (-1)^k)2^{-k}$. Вычисли r_{k+1}/r_k и $r_k^{1/k}$. Какой из двух тестов даёт ответ, а какой — нет? Сделай вывод об их сравнительной силе.

ЗАДАЧА 2

Один метод гарантирует $r_{k+1} \leq 0,9r_k$, другой — $r_{k+1} \leq r_k^2$ при $r_0 = 0,5$. Сколько итераций нужно каждому из них для достижения точности 10^{-15} ?

ЗАДАЧА 3

Рассмотрим метод простой итерации $x_{k+1} = x_k - \tau(Ax_k - b)$ для системы $Ax = b$ с

$$A = \begin{pmatrix} 100 & 1 \\ 1 & 2 \end{pmatrix}.$$

1. Покажи, что ошибка удовлетворяет $x_{k+1} - x^* = (I - \tau A)(x_k - x^*)$, и выпиши знаменатель линейной сходимости $q(\tau) = \max_i |1 - \tau \lambda_i(A)|$.
2. Найди собственные значения A и число обусловленности $\kappa(A)$. Вычисли оптимальный шаг $\tau^* = 2/(\mu + L)$ и соответствующий q^* . Сколько итераций нужно для точности 10^{-6} ?
3. Возьми диагональный предобуславливатель $P = \text{diag}(A) = \text{diag}(100, 2)$ и выпиши матрицу $P^{-1}A$. Найди её собственные значения и $\kappa(P^{-1}A)$.
4. Посчитай q^* для предобусловленного метода и число итераций до той же точности. Во сколько раз ускорился метод? Во что обошлось одно применение P^{-1} ?

ЗАДАЧА 4

Пусть $A = A^T$, её собственные значения удовлетворяют $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$, а собственные векторы q_i ортонормированы. Рассмотрим степенной метод $x_{k+1} = Ax_k / \|Ax_k\|$.

1. Разложив $x_0 = \sum_i c_i q_i$ (считаем $c_1 \neq 0$), покажи, что для угла θ_k между x_k и q_1 выполнено

$$\tan \theta_k \leq \left| \frac{\lambda_2}{\lambda_1} \right|^k \tan \theta_0.$$

Какой это тип сходимости? Что произойдёт, если $\lambda_1 = -\lambda_2$? А если $c_1 = 0$?

2. Сколько итераций степенного метода нужно для достижения точности ε , если $|\lambda_2/\lambda_1| = 0,99$?
3. Какова сложность одной итерации для плотной матрицы и для разрежённой матрицы с nnz ненулевыми элементами?

ЗАДАЧА 5

В обозначениях параграфа о PageRank:

1. Проверь, что G стохастична по столбцам и что $\lambda_1(G) = 1$, причём соответствующий собственный вектор и есть искомый вектор рангов π .
2. Проверь, что степенной метод сохраняет условие $\sum_i (x_k)_i = 1$: если x_k — вероятностный вектор, то и Gx_k тоже.

3. Матрица G плотная, а $n \sim 10^9$. Покажи, как вычислить Gx за $O(\text{nnz}(P) + n)$ операций, ни разу не формируя G явно.
4. Известно, что $|\lambda_2(G)| \leq \alpha$. Сколько итераций степенного метода нужно для точности 10^{-6} ? Как изменится это число при $\alpha = 0,99$ и почему на практике не берут α , близкое к единице?